



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ N 1. ЭНИГМА

Название: «Разработка программы шифрования файлов на основе машины Enigma»

Дисциплина: «Защита информации»

Студент ИУ7-72Б
(Группа)

(Подпись, дата)

Ю.А. Попов
(И. О. Фамилия)

Преподаватель

(Подпись, дата)

Ю.С. Руденкова
(И. О. Фамилия)

Москва, 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Анализ предметной области	4
1.1 Принцип работы машины Enigma	4
1.2 Обратимость преобразования	4
2 Проектирование и реализация программы	6
2.1 Структура данных	6
2.2 Преобразование байта	6
2.3 Интерфейс командной строки и обработка файла	7
3 Тестирование	8
3.1 Методика тестирования	8
3.2 Результаты тестирования	8
ЗАКЛЮЧЕНИЕ	10
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	11

ВВЕДЕНИЕ

Роторные шифровальные машины стали важным этапом развития криптографических средств. В машине Enigma результат преобразования зависел не только от нажатой клавиши, но и от текущего положения роторов. После каждого символа состояние машины изменялось, поэтому одинаковые входные символы могли иметь разные шифртексты.

Цель работы — разработать консольную программу на языке Go для обратимого шифрования файлов на основе 256-контактного обобщения механизма Enigma.

Для достижения цели необходимо решить следующие задачи:

- 1) изучить путь сигнала и механизм вращения роторов Enigma;
- 2) спроектировать роторную машину для полного алфавита байтов от 0 до 255;
- 3) реализовать обработку файлов без загрузки всего содержимого в память;
- 4) реализовать интерфейс командной строки с помощью библиотеки Cobra;
- 5) проверить обратимость преобразования на бинарных данных.

1 Анализ предметной области

1.1 Принцип работы машины Enigma

Историческая Enigma использовала набор роторов, отражатель и, в большинстве конфигураций, штекерную панель. Сигнал проходил через роторы справа налево, затем через отражатель и возвращался через те же роторы в обратном направлении. Благодаря отражателю преобразование было симметричным: при одинаковом начальном состоянии шифрование и расшифрование выполнялись одной и той же операцией [1].

Перед обработкой символа правый ротор выполнял шаг. При достижении точки переключения он перемещал средний ротор. Средний ротор, достигнув собственной точки переключения, одновременно перемещался сам и перемещал левый ротор. Такое поведение называют двойным шагом [2].

Историческая Enigma I имела алфавит из 26 букв, поэтому она не могла непосредственно обработать произвольный файл. Для выполнения требования по шифрованию любых байтов в работе используется обобщенная схема с алфавитом мощности 256. При этом сохраняются структура пути сигнала, отражатель, смещение роторов и двойной шаг, но таблица проводки каждого ротора содержит 256 значений.

1.2 Обратимость преобразования

Каждый ротор задает биективное отображение множества байтов. Пусть R_l , R_m и R_r обозначают преобразования левого, среднего и правого роторов, а U — отражатель. Тогда преобразование одного байта выражается формулой

$$E(x) = R_r^{-1} \left(R_m^{-1} \left(R_l^{-1} \left(U \left(R_l \left(R_m \left(R_r(x) \right) \right) \right) \right) \right) \right). \quad (1.1)$$

Здесь верхний индекс (-1) обозначает обратное отображение. Отражатель попарно связывает значения и является собственной обратной функцией: $U^{-1} =$

U . Поэтому для одного и того же положения роторов выполняется равенство $E(E(x)) = x$.

Положение роторов является частью состояния машины. В программе оно задано структурой `Position` с полями `Left`, `Middle` и `Right` типа `byte`. Начальные значения передаются в CLI строкой `left,middle,right`, например `12,34,56`.

2 Проектирование и реализация программы

2.1 Структура данных

Для каждого ротора хранятся прямая и обратная таблицы проводки, точка переключения и текущая позиция. Обратная таблица позволяет получить обратное преобразование за постоянное время и не выполнять поиск по массиву. Состав основных структур приведен в Таблица 2.1.

Таблица 2.1 — Основные структуры программы

Сущность	Поле	Назначение
Ротор	wiring	Прямая перестановка 256 значений байта.
Ротор	reverse	Обратная перестановка для пути сигнала после отражателя.
Ротор	notch	Позиция, при которой запускается перемещение соседнего ротора.
Ротор	position	Текущее смещение проводки.
Машина	reflector	Попарная перестановка, меняющая направление сигнала.
Позиция	Left, Middle, Right	Начальные положения трех роторов.

Таблицы проводки формируются при создании машины. Массив сначала заполняется значениями от 0 до 255, после чего детерминированно перемещается. Фиксированное начальное значение генератора обеспечивает получение одной и той же проводки для каждого из трех роторов при каждом запуске программы. Оно не является секретным ключом.

2.2 Преобразование байта

Метод `Crypt` сначала перемещает роторы, затем проводит байт через прямые таблицы, отражатель и обратные таблицы. Фрагмент реализации приведен в Листинг 2.1.

```
1 func (machine *Enigma) Crypt(value byte) byte {
2     machine.stepRotors()
3     encoded := machine.right.forward(value)
4     encoded = machine.middle.forward(encoded)
5     encoded = machine.left.forward(encoded)
6     encoded = machine.reflector[encoded]
```

```
7     encoded = machine.left.backward(encoded)
8     encoded = machine.middle.backward(encoded)
9     return machine.right.backward(encoded)
10 }
```

Листинг 2.1 — Преобразование одного байта в машине Enigma

Смещение ротора учитывается перед обращением к таблице и компенсируется после получения значения. Текущая позиция увеличивается по модулю 256. Таким образом, в обработке участвуют все возможные значения типа `byte`, включая `0x00` и `0xFF`.

2.3 Интерфейс командной строки и обработка файла

Для интерфейса командной строки использована библиотека `Cobra`. Она строит приложение из команд и флагов, автоматически формирует справку и поддерживает короткие и длинные формы параметров [3]. Команда `encrypt` принимает пути входного и выходного файлов, а также начальные позиции роторов. Значения по умолчанию равны `input.txt`, `output.txt` и `0,0,0` соответственно.

Файл читается блоками размером 32 KiB. Каждый байт блока передается методу `Crypt`, затем результат записывается через `bufio.Writer`. Буферизация уменьшает количество операций записи в файловый дескриптор; вызов `Flush` в конце сохраняет остаток внутреннего буфера Go [4].

```
1 input file -> reader -> 32 KiB byte buffer -> Enigma.Crypt -> writer -> output file
```

Листинг 2.2 — Поток обработки входного файла

3 Тестирование

3.1 Методика тестирования

Проверка корректности выполнена автоматическими модульными тестами пакета `internal/enigma`. Основным свойством роторной машины является обратимость: новая машина с теми же начальными позициями должна восстановить последовательность, ранее обработанную другой машиной. Для проверки используются три сценария, приведенные в Таблица 3.1.

Таблица 3.1 — Набор автоматических тестов

N	Проверяемое условие	Ожидаемый результат
1	Шифрование и повторное преобразование 512 байтов с позицией 12, 34, 56.	Исходный и расшифрованный массивы совпадают побайтно.
2	Разбор строки позиций 0, 0, 256.	Возвращается ошибка, так как 256 не представляется типом <code>byte</code> .
3	Создание машины с левой позицией 255.	Внутреннее положение левого ротора равно 255 и не изменяется при инициализации.

В первом сценарии массив длиной 512 байтов формируется так, чтобы полный диапазон значений от 0 до 255 встретился два раза. Это исключает проверку только текстовых данных и подтверждает обработку управляющих и непечатаемых байтов, включая `0x00` и `0xFF`.

3.2 Результаты тестирования

Тесты запускаются из корня лабораторной работы. Цель `test` в `Makefile` выполняет тот же набор проверок в подробном режиме.

Команды проверки и сборки имеют вид:

```
1 go test ./...
2 make test
3 make build
```

Листинг 3.1 — Команды тестирования и сборки программы

Все три сценария завершаются успешно. Полученный результат подтверждает обратимость преобразования для бинарных данных, корректность диапазона

пользовательских позиций и сохранение граничного значения байта при создании машины.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы разработана программа шифрования файлов на языке Go. Реализован 256-контактный вариант роторной машины, который использует три ротора, отражатель и механизм двойного шага. В отличие от исторической Enigma I, реализация обрабатывает все значения байта и поэтому применима к произвольным бинарным файлам.

Для работы с программой создан интерфейс командной строки на базе Cobra. Файл обрабатывается потоково блоками по 32 KiB, что исключает необходимость целиком загружать его в оперативную память. Автоматические тесты подтвердили, что повторное применение операции с одинаковыми начальными позициями восстанавливает исходный поток байтов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. National Security Agency. Outline of Enigma Machines [Электронный ресурс]. — 2026. — Режим доступа: https://www.cryptomuseum.com/crypto/enigma/files/enigma_wiring_catalog.pdf.
2. Hamer D. H. Enigma: Actions Involved In Double Stepping [Электронный ресурс]. — 2026. — Режим доступа: https://www.cryptomuseum.com/people/hamer/files/double_stepping.pdf.
3. The Cobra Authors. cobra package [Электронный ресурс]. — 2026. — Режим доступа: <https://pkg.go.dev/github.com/spf13/cobra>.
4. The Go Authors. bufio package [Электронный ресурс]. — 2026. — Режим доступа: <https://pkg.go.dev/bufio>.